SHEMA SQL

```
--
================================================================================
-- INDIAOS MVP DATABASE SCHEMA - PRODUCTION READY (FINAL)
-- PostgreSQL 14+ / Supabase
-- Generated from database_schema.md (locked specification)
--
-- FINAL CORRECTIONS APPLIED:
--  ✓ Critical: citizens.report_count now incremented on report insert
--  ✓ Design: Removed reports_24h/7d/30d (compute on-the-fly for MVP simplicity)
--  ✓ Governance Memory: governance_terms delete prevention added
--  ✓ Foreign keys complete (reports → issues)
--  ✓ INSERT RLS policies (citizens can submit)
--  ✓ Service_role bypass configured
--  ✓ Report reclustering trigger fixed
--  ✓ Redundant policies removed
--
================================================================================


--
================================================================================
-- SECTION 1: ENUM TYPES
--
================================================================================


CREATE TYPE citizen_status_enum AS ENUM (
  'active',
```

SHEMA SQL

```
 'inactive',
 'suspended'
);

CREATE TYPE report_status_enum AS ENUM (
 'submitted',
 'verified',
 'disputed',
 'archived'
);

CREATE TYPE verification_level_enum AS ENUM (
 'phone_verified',
 'aadhaar_verified',
 'government_verified'
);

CREATE TYPE official_status_enum AS ENUM (
 'unacknowledged',
 'acknowledged',
 'in_progress',
 'completed',
 'dismissed'
);

CREATE TYPE reality_status_enum AS ENUM (
 'emerging',
 'verified',
```

SHEMA SQL

```
 'trending',
 'in_progress',
 'partially_resolved',
 'resolved',
 'archived'
);

CREATE TYPE responsibility_type_enum AS ENUM (
 'primary',
 'secondary',
 'supporting'
);

CREATE TYPE evidence_source_type_enum AS ENUM (
 'news_outlet',
 'rti_portal',
 'government_agency',
 'court',
 'academic',
 'ngo',
 'sensor',
 'other'
);

CREATE TYPE evidence_type_enum AS ENUM (
 'news_article',
 'rti_response',
 'government_document',
```

SHEMA SQL

```
  'court_order',
  'photo',
  'video',
  'audio',
  'sensor_data',
  'other'
);

CREATE TYPE governance_level_enum AS ENUM (
  'national',
  'state',
  'district',
  'city',
  'ward'
);

CREATE TYPE entity_type_enum AS ENUM (
  'government',
  'department',
  'agency',
  'municipality',
  'other'
);

CREATE TYPE update_type_enum AS ENUM (
  'tender_approved',
  'budget_released',
  'work_started',
```

SHEMA SQL

```
  'progress_update',
  'milestone',
  'completion',
  'delay'
);

CREATE TYPE outcome_type_enum AS ENUM (
  'funds_released',
  'tender_approved',
  'project_started',
  'project_completed',
  'milestone',
  'policy_change',
  'other'
);

CREATE TYPE lifecycle_event_type_enum AS ENUM (
  'created',
  'verified',
  'trending',
  'in_progress',
  'partially_resolved',
  'resolved',
  'reopened',
  'archived'
);

CREATE TYPE trigger_type_enum AS ENUM (
```

SHEMA SQL

```
  'ai_clustering',
  'momentum_threshold',
  'government_action',
  'evidence_update',
  'rgs_reopen',
  'citizen_flag',
  'admin_override'
);

CREATE TYPE claimed_status_enum AS ENUM (
  'acknowledged',
  'in_progress',
  'completed',
  'dismissed'
);

CREATE TYPE message_type_enum AS ENUM (
  'text',
  'photo',
  'video',
  'audio',
  'document'
);

CREATE TYPE message_status_enum AS ENUM (
  'sent',
  'delivered',
  'read',
```

SHEMA SQL

```
 'failed'
);


--
================================================================================
======
-- SECTION 2: BASE TABLES (No FK dependencies)
--
================================================================================
======


-- TIER 1: CITIZENS
CREATE TABLE citizens (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 phone_hash VARCHAR(128) UNIQUE NOT NULL,
 phone_verified_at TIMESTAMPTZ,
 verification_level verification_level_enum NOT NULL DEFAULT 'phone_verified',
 status citizen_status_enum NOT NULL DEFAULT 'active',
 status_changed_at TIMESTAMPTZ,
 created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
 last_activity_at TIMESTAMPTZ,
 report_count INT DEFAULT 0,

 CONSTRAINT citizens_phone_hash_length CHECK (char_length(phone_hash) = 128),
 CONSTRAINT citizens_report_count_nonnegative CHECK (report_count >= 0)
);


CREATE INDEX idx_citizens_active ON citizens(status) WHERE status = 'active';
```

SHEMA SQL

```
-- TIER 3: CATEGORIES
CREATE TABLE categories (
 id SERIAL PRIMARY KEY,
 name VARCHAR(128) UNIQUE NOT NULL,
 parent_category_id INT,
 description TEXT,
 sla_days_benchmark SMALLINT,
 created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),

 CONSTRAINT fk_categories_parent FOREIGN KEY (parent_category_id)
  REFERENCES categories(id) ON DELETE SET NULL,
 CONSTRAINT categories_sla_days_nonnegative CHECK (sla_days_benchmark >= 0)
);

-- TIER 3: LOCATIONS
CREATE TABLE locations (
 id SERIAL PRIMARY KEY,
 name VARCHAR(128) NOT NULL,
 location_type VARCHAR(64),
 parent_location_id INT,
 latitude NUMERIC(9,6),
 longitude NUMERIC(9,6),
 population INT,
 created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),

 CONSTRAINT fk_locations_parent FOREIGN KEY (parent_location_id)
  REFERENCES locations(id) ON DELETE SET NULL,
 CONSTRAINT locations_latitude_range CHECK (latitude BETWEEN -90 AND 90),
```

SHEMA SQL

```
 CONSTRAINT locations_longitude_range CHECK (longitude BETWEEN -180 AND 180),
 CONSTRAINT locations_population_nonnegative CHECK (population >= 0)
);

-- TIER 4: GOVERNANCE_ENTITIES
CREATE TABLE governance_entities (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 name VARCHAR(255) NOT NULL,
 entity_type entity_type_enum NOT NULL,
 governance_level governance_level_enum NOT NULL,
 location_id INT NOT NULL,
 parent_entity_id UUID,
 contact_email VARCHAR(255),
 contact_phone VARCHAR(20),
 website_url VARCHAR(255),
 created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),

 CONSTRAINT fk_governance_entities_location FOREIGN KEY (location_id)
  REFERENCES locations(id) ON DELETE RESTRICT,
 CONSTRAINT fk_governance_entities_parent FOREIGN KEY (parent_entity_id)
  REFERENCES governance_entities(id) ON DELETE SET NULL
);

CREATE INDEX idx_governance_entities_level ON
governance_entities(governance_level);

-- TIER 5: EVIDENCE_SOURCES
CREATE TABLE evidence_sources (
```

SHEMA SQL

```
 id SERIAL PRIMARY KEY,
 source_name VARCHAR(255) UNIQUE NOT NULL,
 source_type evidence_source_type_enum NOT NULL,
 base_credibility_score NUMERIC(3,2) NOT NULL,
 url_pattern VARCHAR(512),
 is_verified BOOLEAN DEFAULT FALSE,
 created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),

 CONSTRAINT evidence_sources_credibility_range CHECK (base_credibility_score
BETWEEN 0 AND 1)
);

--
================================================================================
======
-- SECTION 3: REPORTS & MEDIA (Tier 1, FK to base tables)
--
================================================================================
======

CREATE TABLE reports (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 citizen_id UUID NOT NULL,
 issue_id UUID,
 category_id INT NOT NULL,
 location_id INT NOT NULL,
 title VARCHAR(255) NOT NULL,
 description TEXT NOT NULL,
 latitude NUMERIC(9,6) NOT NULL,
```

SHEMA SQL

```
 longitude NUMERIC(9,6) NOT NULL,
 status report_status_enum NOT NULL DEFAULT 'submitted',
 is_duplicate BOOLEAN DEFAULT FALSE,
 ai_clustering_decision JSONB,
 created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
 verified_at TIMESTAMPTZ,
 archived BOOLEAN DEFAULT FALSE,

 CONSTRAINT fk_reports_citizen FOREIGN KEY (citizen_id)
  REFERENCES citizens(id) ON DELETE RESTRICT,
 CONSTRAINT fk_reports_category FOREIGN KEY (category_id)
  REFERENCES categories(id) ON DELETE RESTRICT,
 CONSTRAINT fk_reports_location FOREIGN KEY (location_id)
  REFERENCES locations(id) ON DELETE RESTRICT,
 CONSTRAINT reports_latitude_range CHECK (latitude BETWEEN -90 AND 90),
 CONSTRAINT reports_longitude_range CHECK (longitude BETWEEN -180 AND 180)
);

CREATE INDEX idx_reports_location_created ON reports(location_id, created_at DESC)
WHERE archived = false;
CREATE INDEX idx_reports_issue ON reports(issue_id);
CREATE INDEX idx_reports_citizen ON reports(citizen_id);

CREATE TABLE reports_media (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 report_id UUID NOT NULL,
 file_path VARCHAR(512) UNIQUE NOT NULL,
 media_type message_type_enum NOT NULL,
```

SHEMA SQL

```
 file_size_bytes BIGINT NOT NULL,
 original_filename VARCHAR(255),
 mime_type VARCHAR(64),
 uploaded_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
 archived BOOLEAN DEFAULT FALSE,

 CONSTRAINT fk_reports_media_report FOREIGN KEY (report_id)
  REFERENCES reports(id) ON DELETE CASCADE,
 CONSTRAINT reports_media_file_size_positive CHECK (file_size_bytes > 0)
);

CREATE INDEX idx_reports_media_report ON reports_media(report_id);


--
============================================================================
======
-- SECTION 4: BOT CONVERSATIONS (Tier 1)
--
============================================================================
======

CREATE TABLE bot_conversations (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 citizen_id UUID NOT NULL,
 message_id VARCHAR(64) UNIQUE NOT NULL,
 direction VARCHAR(10) NOT NULL CHECK (direction IN ('inbound', 'outbound')),
 message_text TEXT,
 message_type message_type_enum NOT NULL,
 timestamp TIMESTAMPTZ NOT NULL,
```

SHEMA SQL

```
 status message_status_enum NOT NULL,
 associated_report_id UUID,
 created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),

 CONSTRAINT fk_bot_conversations_citizen FOREIGN KEY (citizen_id)
  REFERENCES citizens(id) ON DELETE RESTRICT,
 CONSTRAINT fk_bot_conversations_report FOREIGN KEY (associated_report_id)
  REFERENCES reports(id) ON DELETE SET NULL
);

CREATE INDEX idx_bot_conversations_citizen ON bot_conversations(citizen_id);


--
================================================================================
------
-- SECTION 5: ISSUES & UPDATES (Tier 2)
--
================================================================================
------

CREATE TABLE issues (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 title VARCHAR(255) NOT NULL,
 description TEXT NOT NULL,
 category_id INT NOT NULL,
 location_id INT NOT NULL,
 latitude NUMERIC(9,6),
 longitude NUMERIC(9,6),
 official_status official_status_enum NOT NULL DEFAULT 'unacknowledged',
```

SHEMA SQL

```
 reality_status reality_status_enum NOT NULL DEFAULT 'emerging',
 confidence_score NUMERIC(3,2) NOT NULL DEFAULT 0.00,
 momentum_score NUMERIC(3,2) NOT NULL DEFAULT 0.00,
 report_count INT DEFAULT 0,
 evidence_count INT DEFAULT 0,
 created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
 updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
 last_activity_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
 is_active BOOLEAN DEFAULT TRUE,

 CONSTRAINT fk_issues_category FOREIGN KEY (category_id)
  REFERENCES categories(id) ON DELETE RESTRICT,
 CONSTRAINT fk_issues_location FOREIGN KEY (location_id)
  REFERENCES locations(id) ON DELETE RESTRICT,
 CONSTRAINT issues_latitude_range CHECK (latitude BETWEEN -90 AND 90),
 CONSTRAINT issues_longitude_range CHECK (longitude BETWEEN -180 AND 180),
 CONSTRAINT issues_confidence_range CHECK (confidence_score BETWEEN 0 AND
1),
 CONSTRAINT issues_momentum_range CHECK (momentum_score BETWEEN 0 AND
1),
 CONSTRAINT issues_report_count_nonnegative CHECK (report_count >= 0),
 CONSTRAINT issues_evidence_count_nonnegative CHECK (evidence_count >= 0)
);

CREATE INDEX idx_issues_confidence_location ON issues(confidence_score DESC,
location_id) WHERE is_active = true;
CREATE INDEX idx_issues_momentum ON issues(momentum_score DESC) WHERE
is_active = true;
CREATE INDEX idx_issues_active ON issues(is_active);
```

SHEMA SQL

```
-- [CRITICAL FIX] Add FK from reports to issues after issues table exists
ALTER TABLE reports
ADD CONSTRAINT fk_reports_issue FOREIGN KEY (issue_id)
  REFERENCES issues(id) ON DELETE SET NULL;

CREATE TABLE issue_updates (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  issue_id UUID NOT NULL,
  governance_entity_id UUID,
  update_type update_type_enum NOT NULL,
  title VARCHAR(255) NOT NULL,
  description TEXT NOT NULL,
  progress_percent SMALLINT,
  evidence_id UUID,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  archived BOOLEAN DEFAULT FALSE,

  CONSTRAINT fk_issue_updates_issue FOREIGN KEY (issue_id)
    REFERENCES issues(id) ON DELETE CASCADE,
  CONSTRAINT fk_issue_updates_governance_entity FOREIGN KEY
(governance_entity_id)
    REFERENCES governance_entities(id) ON DELETE SET NULL,
  CONSTRAINT issue_updates_progress_range CHECK (progress_percent BETWEEN 0
AND 100)
);

CREATE INDEX idx_issue_updates_issue ON issue_updates(issue_id, created_at
DESC);
```

SHEMA SQL

```
--
================================================================================
-- SECTION 6: GOVERNANCE (Tier 4)
--
================================================================================

CREATE TABLE governance_terms (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 governance_entity_id UUID NOT NULL,
 term_start_date DATE NOT NULL,
 term_end_date DATE,
 ruling_party_name VARCHAR(128),
 chief_executive_name VARCHAR(255),
 notes TEXT,
 created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),

 CONSTRAINT fk_governance_terms_entity FOREIGN KEY (governance_entity_id)
  REFERENCES governance_entities(id) ON DELETE CASCADE,
 CONSTRAINT governance_terms_date_order CHECK (term_end_date IS NULL OR
term_end_date >= term_start_date)
);

CREATE INDEX idx_governance_terms_entity ON
governance_terms(governance_entity_id);

CREATE TABLE issue_governance_entities (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

SHEMA SQL

```
 issue_id UUID NOT NULL,
 governance_entity_id UUID NOT NULL,
 responsibility_type responsibility_type_enum NOT NULL,
 assigned_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
 acknowledgment_date DATE,
 acknowledgment_text TEXT,
 created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),

 CONSTRAINT fk_issue_gov_entities_issue FOREIGN KEY (issue_id)
  REFERENCES issues(id) ON DELETE CASCADE,
 CONSTRAINT fk_issue_gov_entities_entity FOREIGN KEY (governance_entity_id)
  REFERENCES governance_entities(id) ON DELETE CASCADE,
 CONSTRAINT uc_issue_gov_entities UNIQUE (issue_id, governance_entity_id)
);

CREATE INDEX idx_issue_gov_entity_entity ON
issue_governance_entities(governance_entity_id);
CREATE INDEX idx_issue_gov_entity_issue ON issue_governance_entities(issue_id);

--
================================================================
======
-- SECTION 7: EVIDENCE & CLAIMS (Tier 5)
--
================================================================
======

CREATE TABLE evidence (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

SHEMA SQL

```
 issue_id UUID NOT NULL,
 evidence_source_id INT NOT NULL,
 evidence_url VARCHAR(512) NOT NULL,
 evidence_title VARCHAR(255) NOT NULL,
 evidence_type evidence_type_enum NOT NULL,
 ers_score NUMERIC(3,2) NOT NULL DEFAULT 0.00,
 extracted_text TEXT,
 semantic_match_score NUMERIC(3,2) DEFAULT 0.00,
 published_at DATE,
 source_name VARCHAR(255),
 author VARCHAR(255),
 is_disputed BOOLEAN DEFAULT FALSE,
 created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),

 CONSTRAINT fk_evidence_issue FOREIGN KEY (issue_id)
  REFERENCES issues(id) ON DELETE CASCADE,
 CONSTRAINT fk_evidence_source FOREIGN KEY (evidence_source_id)
  REFERENCES evidence_sources(id) ON DELETE RESTRICT,
 CONSTRAINT evidence_ers_range CHECK (ers_score BETWEEN 0 AND 1),
 CONSTRAINT evidence_semantic_range CHECK (semantic_match_score BETWEEN 0
AND 1)
);

CREATE INDEX idx_evidence_issue ON evidence(issue_id);
CREATE INDEX idx_evidence_source ON evidence(evidence_source_id);

CREATE TABLE resolution_claims (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

SHEMA SQL

```
 issue_id UUID NOT NULL,
 governance_entity_id UUID NOT NULL,
 claim_number SMALLINT NOT NULL,
 claimed_status claimed_status_enum NOT NULL,
 claim_date DATE NOT NULL,
 claim_evidence_url VARCHAR(512),
 claim_evidence_id UUID,
 reality_status_at_claim reality_status_enum NOT NULL,
 current_reality_status reality_status_enum NOT NULL,
 is_valid BOOLEAN,
 reason_if_invalid TEXT,
 created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),

 CONSTRAINT fk_resolution_claims_issue FOREIGN KEY (issue_id)
  REFERENCES issues(id) ON DELETE CASCADE,
 CONSTRAINT fk_resolution_claims_entity FOREIGN KEY (governance_entity_id)
  REFERENCES governance_entities(id) ON DELETE CASCADE,
 CONSTRAINT fk_resolution_claims_evidence FOREIGN KEY (claim_evidence_id)
  REFERENCES evidence(id) ON DELETE SET NULL,
 CONSTRAINT uc_resolution_claims UNIQUE (issue_id, governance_entity_id,
claim_date)
);

CREATE INDEX idx_resolution_claims_entity ON
resolution_claims(governance_entity_id, is_valid);
CREATE INDEX idx_resolution_claims_issue ON resolution_claims(issue_id);
```

SHEMA SQL

```
--
================================================================================
-- SECTION 8: OUTCOMES & LIFECYCLE (Tier 6)
--
================================================================================

CREATE TABLE issue_outcomes (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 issue_id UUID NOT NULL,
 governance_entity_id UUID NOT NULL,
 outcome_type outcome_type_enum NOT NULL,
 amount_inr BIGINT,
 outcome_date DATE NOT NULL,
 outcome_description TEXT NOT NULL,
 confirming_evidence_id UUID,
 confirmed_date DATE,
 beneficiaries_count INT,
 created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),

 CONSTRAINT fk_issue_outcomes_issue FOREIGN KEY (issue_id)
  REFERENCES issues(id) ON DELETE CASCADE,
 CONSTRAINT fk_issue_outcomes_entity FOREIGN KEY (governance_entity_id)
  REFERENCES governance_entities(id) ON DELETE CASCADE,
 CONSTRAINT fk_issue_outcomes_evidence FOREIGN KEY (confirming_evidence_id)
  REFERENCES evidence(id) ON DELETE SET NULL,
 CONSTRAINT issue_outcomes_amount_positive CHECK (amount_inr > 0),
```

SHEMA SQL

```
 CONSTRAINT issue_outcomes_beneficiaries_nonnegative CHECK
(beneficiaries_count >= 0)

);


CREATE INDEX idx_issue_outcomes_issue ON issue_outcomes(issue_id);

CREATE INDEX idx_issue_outcomes_entity ON issue_outcomes(governance_entity_id);


CREATE TABLE issue_lifecycle_events (

 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

 issue_id UUID NOT NULL,

 event_type lifecycle_event_type_enum NOT NULL,

 triggered_by trigger_type_enum NOT NULL,

 triggered_by_entity_id UUID,

 confidence_score_at_event NUMERIC(3,2),

 momentum_score_at_event NUMERIC(3,2),

 report_count_at_event INT,

 reason TEXT NOT NULL,

 evidence_snapshot JSONB,

 created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),


 CONSTRAINT fk_lifecycle_events_issue FOREIGN KEY (issue_id)

  REFERENCES issues(id) ON DELETE CASCADE,

 CONSTRAINT fk_lifecycle_events_entity FOREIGN KEY (triggered_by_entity_id)

  REFERENCES governance_entities(id) ON DELETE SET NULL,

 CONSTRAINT lifecycle_events_confidence_range CHECK (confidence_score_at_event
BETWEEN 0 AND 1),

 CONSTRAINT lifecycle_events_momentum_range CHECK
(momentum_score_at_event BETWEEN 0 AND 1)

);
```

SHEMA SQL

```
CREATE INDEX idx_lifecycle_issue_date ON issue_lifecycle_events(issue_id, created_at
DESC);


-- Link issue_updates to evidence after evidence table exists
ALTER TABLE issue_updates
  ADD CONSTRAINT fk_issue_updates_evidence FOREIGN KEY (evidence_id)
    REFERENCES evidence(id) ON DELETE SET NULL;


--
================================================================================
======
-- SECTION 9: TRIGGER FUNCTIONS
--
================================================================================
======


-- Prevent hard deletion of citizens
CREATE FUNCTION prevent_citizen_hard_delete()
  RETURNS TRIGGER AS $$
BEGIN
  RAISE EXCEPTION 'Citizens cannot be hard-deleted. Use UPDATE status=inactive
instead.';
END;
$$ LANGUAGE plpgsql;


-- Prevent hard deletion of reports
CREATE FUNCTION prevent_report_hard_delete()
  RETURNS TRIGGER AS $$
BEGIN
```

SHEMA SQL

```
  RAISE EXCEPTION 'Reports cannot be hard-deleted. Use UPDATE archived=true instead.';
END;
$$ LANGUAGE plpgsql;

-- Prevent updates to governance_terms (Governance Memory: immutable)
CREATE FUNCTION prevent_governance_terms_update()
  RETURNS TRIGGER AS $$
BEGIN
  RAISE EXCEPTION 'governance_terms is immutable. Insert new term instead of updating.';
END;
$$ LANGUAGE plpgsql;

-- [CRITICAL FIX] Prevent deletion of governance_terms (Governance Memory: permanent)
CREATE FUNCTION prevent_governance_terms_delete()
  RETURNS TRIGGER AS $$
BEGIN
  RAISE EXCEPTION 'governance_terms cannot be deleted. Governance history is permanent.';
END;
$$ LANGUAGE plpgsql;

-- Prevent updates to issue_lifecycle_events
CREATE FUNCTION prevent_lifecycle_events_update()
  RETURNS TRIGGER AS $$
BEGIN
  RAISE EXCEPTION 'issue_lifecycle_events is immutable and append-only.';
```

```
SHEMA SQL

END;
$$ LANGUAGE plpgsql;


-- Prevent deletion of issue_lifecycle_events
CREATE FUNCTION prevent_lifecycle_events_delete()
 RETURNS TRIGGER AS $$
BEGIN
 RAISE EXCEPTION 'issue_lifecycle_events cannot be deleted.';
END;
$$ LANGUAGE plpgsql;


-- [CRITICAL FIX] Update issues counters on report link/move
-- Handles INSERT (new report), UPDATE (reclustering), and NULL moves
CREATE FUNCTION update_issue_on_report_link()
 RETURNS TRIGGER AS $$
BEGIN
 -- Handle report INSERT (new report linked to issue)
 IF TG_OP = 'INSERT' THEN
  IF NEW.issue_id IS NOT NULL THEN
   UPDATE issues
   SET report_count = report_count + 1,
     updated_at = NOW(),
     last_activity_at = NOW()
   WHERE id = NEW.issue_id;
  END IF;
  RETURN NEW;

 -- Handle report UPDATE (report moved to different issue or unlinked)
```

SHEMA SQL

```
 ELSIF TG_OP = 'UPDATE' THEN
  -- Decrement old issue if report was previously linked
  -- AND now linked to a different issue or unlinked
  IF OLD.issue_id IS NOT NULL AND OLD.issue_id IS DISTINCT FROM NEW.issue_id
THEN
    UPDATE issues
    SET report_count = GREATEST(report_count - 1, 0),
      updated_at = NOW(),
      last_activity_at = NOW()
    WHERE id = OLD.issue_id;
  END IF;

  -- Increment new issue if report newly linked
  -- AND wasn't linked before or linked to different issue
  IF NEW.issue_id IS NOT NULL AND OLD.issue_id IS DISTINCT FROM NEW.issue_id
THEN
    UPDATE issues
    SET report_count = report_count + 1,
      updated_at = NOW(),
      last_activity_at = NOW()
    WHERE id = NEW.issue_id;
  END IF;

  RETURN NEW;
 END IF;

 RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

SHEMA SQL

```
-- Update issues counters on evidence insert
CREATE FUNCTION update_issue_on_evidence_insert()
 RETURNS TRIGGER AS $$
BEGIN
 UPDATE issues
 SET evidence_count = evidence_count + 1,
    updated_at = NOW(),
    last_activity_at = NOW()
 WHERE id = NEW.issue_id;
 RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- Update issues on issue_update insert
CREATE FUNCTION update_issue_on_issue_update_insert()
 RETURNS TRIGGER AS $$
BEGIN
 UPDATE issues
 SET updated_at = NOW(),
    last_activity_at = NOW()
 WHERE id = NEW.issue_id;
 RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- Update issues on resolution_claim insert
CREATE FUNCTION update_issue_on_resolution_claim_insert()
```

SHEMA SQL

```
 RETURNS TRIGGER AS $$
BEGIN
 UPDATE issues
 SET updated_at = NOW(),
    last_activity_at = NOW()
 WHERE id = NEW.issue_id;
 RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- Update issues on outcome insert
CREATE FUNCTION update_issue_on_outcome_insert()
 RETURNS TRIGGER AS $$
BEGIN
 UPDATE issues
 SET updated_at = NOW(),
    last_activity_at = NOW()
 WHERE id = NEW.issue_id;
 RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- [CRITICAL FIX] Update citizen report_count AND last_activity_at on report insert
CREATE FUNCTION update_citizen_activity_on_report()
 RETURNS TRIGGER AS $$
BEGIN
 UPDATE citizens
 SET
```

SHEMA SQL

```
   last_activity_at = NOW(),
   report_count = report_count + 1
 WHERE id = NEW.citizen_id;
 RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- Update citizen last_activity_at on bot message
CREATE FUNCTION update_citizen_activity_on_bot_message()
 RETURNS TRIGGER AS $$
BEGIN
 UPDATE citizens
 SET last_activity_at = NOW()
 WHERE id = NEW.citizen_id;
 RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- Update issues.updated_at when official_status changes
CREATE FUNCTION update_issue_timestamp_on_status_change()
 RETURNS TRIGGER AS $$
BEGIN
 NEW.updated_at = NOW();
 RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

SHEMA SQL

```
--
================================================================================
-- SECTION 10: TRIGGERS
--
================================================================================

-- Hard delete prevention
CREATE TRIGGER trg_prevent_citizen_delete
  BEFORE DELETE ON citizens
  FOR EACH ROW
  EXECUTE FUNCTION prevent_citizen_hard_delete();

CREATE TRIGGER trg_prevent_report_delete
  BEFORE DELETE ON reports
  FOR EACH ROW
  EXECUTE FUNCTION prevent_report_hard_delete();

-- Governance Memory: immutability enforcement
CREATE TRIGGER trg_prevent_governance_terms_update
  BEFORE UPDATE ON governance_terms
  FOR EACH ROW
  EXECUTE FUNCTION prevent_governance_terms_update();

-- [CRITICAL FIX] Governance Memory: deletion prevention
CREATE TRIGGER trg_prevent_governance_terms_delete
  BEFORE DELETE ON governance_terms
  FOR EACH ROW
```

SHEMA SQL

```
  EXECUTE FUNCTION prevent_governance_terms_delete();


CREATE TRIGGER trg_prevent_lifecycle_events_update
  BEFORE UPDATE ON issue_lifecycle_events
  FOR EACH ROW
  EXECUTE FUNCTION prevent_lifecycle_events_update();


CREATE TRIGGER trg_prevent_lifecycle_events_delete
  BEFORE DELETE ON issue_lifecycle_events
  FOR EACH ROW
  EXECUTE FUNCTION prevent_lifecycle_events_delete();


-- Counter maintenance
CREATE TRIGGER trg_update_issue_on_report_link
  AFTER INSERT OR UPDATE OF issue_id ON reports
  FOR EACH ROW
  EXECUTE FUNCTION update_issue_on_report_link();


CREATE TRIGGER trg_update_issue_on_evidence_insert
  AFTER INSERT ON evidence
  FOR EACH ROW
  EXECUTE FUNCTION update_issue_on_evidence_insert();


CREATE TRIGGER trg_update_issue_on_issue_update_insert
  AFTER INSERT ON issue_updates
  FOR EACH ROW
  EXECUTE FUNCTION update_issue_on_issue_update_insert();
```

SHEMA SQL

```
CREATE TRIGGER trg_update_issue_on_resolution_claim_insert
 AFTER INSERT ON resolution_claims
 FOR EACH ROW
 EXECUTE FUNCTION update_issue_on_resolution_claim_insert();

CREATE TRIGGER trg_update_issue_on_outcome_insert
 AFTER INSERT ON issue_outcomes
 FOR EACH ROW
 EXECUTE FUNCTION update_issue_on_outcome_insert();

-- Activity tracking
CREATE TRIGGER trg_update_citizen_activity_on_report
 AFTER INSERT ON reports
 FOR EACH ROW
 EXECUTE FUNCTION update_citizen_activity_on_report();

CREATE TRIGGER trg_update_citizen_activity_on_bot_message
 AFTER INSERT ON bot_conversations
 FOR EACH ROW
 EXECUTE FUNCTION update_citizen_activity_on_bot_message();

CREATE TRIGGER trg_update_issue_timestamp_on_status_change
 BEFORE UPDATE ON issues
 FOR EACH ROW
 EXECUTE FUNCTION update_issue_timestamp_on_status_change();

--
================================================================
======
```

SHEMA SQL

```
-- SECTION 11: ROW-LEVEL SECURITY (RLS) - ENABLE
--
================================================================================


-- Enable RLS on all tables
ALTER TABLE citizens ENABLE ROW LEVEL SECURITY;
ALTER TABLE reports ENABLE ROW LEVEL SECURITY;
ALTER TABLE reports_media ENABLE ROW LEVEL SECURITY;
ALTER TABLE bot_conversations ENABLE ROW LEVEL SECURITY;
ALTER TABLE issues ENABLE ROW LEVEL SECURITY;
ALTER TABLE issue_updates ENABLE ROW LEVEL SECURITY;
ALTER TABLE categories ENABLE ROW LEVEL SECURITY;
ALTER TABLE locations ENABLE ROW LEVEL SECURITY;
ALTER TABLE governance_entities ENABLE ROW LEVEL SECURITY;
ALTER TABLE governance_terms ENABLE ROW LEVEL SECURITY;
ALTER TABLE issue_governance_entities ENABLE ROW LEVEL SECURITY;
ALTER TABLE evidence_sources ENABLE ROW LEVEL SECURITY;
ALTER TABLE evidence ENABLE ROW LEVEL SECURITY;
ALTER TABLE resolution_claims ENABLE ROW LEVEL SECURITY;
ALTER TABLE issue_outcomes ENABLE ROW LEVEL SECURITY;
ALTER TABLE issue_lifecycle_events ENABLE ROW LEVEL SECURITY;


--
================================================================================
======
-- SECTION 12: RLS POLICIES - CITIZENS TABLE
--
================================================================================
======
```

SHEMA SQL

```
CREATE POLICY "Citizens: Select own row"
  ON citizens FOR SELECT
  USING (auth.uid()::uuid = id);

CREATE POLICY "Admins: Full access citizens"
  ON citizens FOR ALL
  USING (auth.jwt() ->> 'role' = 'admin');


--
================================================================================
------
-- SECTION 13: RLS POLICIES - REPORTS TABLE
--
================================================================================
------

CREATE POLICY "Citizens: Select own reports"
  ON reports FOR SELECT
  USING (citizen_id = auth.uid()::uuid);

CREATE POLICY "Citizens: Insert own reports"
  ON reports FOR INSERT
  WITH CHECK (citizen_id = auth.uid()::uuid);

CREATE POLICY "Admins: Full access reports"
  ON reports FOR ALL
  USING (auth.jwt() ->> 'role' = 'admin');
```

SHEMA SQL

```
--
================================================================================
-- SECTION 14: RLS POLICIES - REPORTS_MEDIA TABLE
--
================================================================================


CREATE POLICY "Citizens: Select own media"
 ON reports_media FOR SELECT
 USING (
  report_id IN (
   SELECT id FROM reports WHERE citizen_id = auth.uid()::uuid
  )
 );

CREATE POLICY "Citizens: Insert own media"
 ON reports_media FOR INSERT
 WITH CHECK (
  report_id IN (
   SELECT id FROM reports WHERE citizen_id = auth.uid()::uuid
  )
 );

CREATE POLICY "Admins: Full access media"
 ON reports_media FOR ALL
 USING (auth.jwt() ->> 'role' = 'admin');
```

SHEMA SQL

```
--
================================================================================
-- SECTION 15: RLS POLICIES - BOT_CONVERSATIONS TABLE
--
================================================================================

CREATE POLICY "Citizens: Select own conversations"
  ON bot_conversations FOR SELECT
  USING (citizen_id = auth.uid()::uuid);

CREATE POLICY "Admins: Full access conversations"
  ON bot_conversations FOR ALL
  USING (auth.jwt() ->> 'role' = 'admin');

-- NOTE: bot_conversations.INSERT restricted to Twilio webhook via service_role.
-- Citizens cannot INSERT directly via authenticated client.

--
================================================================================
-- SECTION 16: RLS POLICIES - ISSUES TABLE (Public)
--
================================================================================

CREATE POLICY "Public: Select all active issues"
  ON issues FOR SELECT
  USING (is_active = true);
```

SHEMA SQL

```
CREATE POLICY "Admins: Full access issues"
  ON issues FOR ALL
  USING (auth.jwt() ->> 'role' = 'admin');

--
================================================================================
-- SECTION 17: RLS POLICIES - ISSUE_UPDATES TABLE (Public)
--
================================================================================

CREATE POLICY "Public: Select all issue updates"
  ON issue_updates FOR SELECT
  USING (archived = false);

CREATE POLICY "Officials: Insert own entity updates"
  ON issue_updates FOR INSERT
  WITH CHECK (
    governance_entity_id = (auth.jwt() ->> 'governance_entity_id')::uuid
  );

CREATE POLICY "Officials: Update own entity updates"
  ON issue_updates FOR UPDATE
  USING (governance_entity_id = (auth.jwt() ->> 'governance_entity_id')::uuid);

CREATE POLICY "Admins: Full access issue_updates"
  ON issue_updates FOR ALL
```

SHEMA SQL

```
 USING (auth.jwt() ->> 'role' = 'admin');


--
================================================================================
-- SECTION 18: RLS POLICIES - CATEGORIES & LOCATIONS (Public)
--
================================================================================


CREATE POLICY "Public: Select categories"
 ON categories FOR SELECT
 USING (true);

CREATE POLICY "Admins: Full access categories"
 ON categories FOR ALL
 USING (auth.jwt() ->> 'role' = 'admin');

CREATE POLICY "Public: Select locations"
 ON locations FOR SELECT
 USING (true);

CREATE POLICY "Admins: Full access locations"
 ON locations FOR ALL
 USING (auth.jwt() ->> 'role' = 'admin');


--
================================================================================
-- SECTION 19: RLS POLICIES - GOVERNANCE ENTITIES & TERMS (Public)
```

SHEMA SQL

```
--
================================================================================

CREATE POLICY "Public: Select governance entities"
 ON governance_entities FOR SELECT
 USING (true);

CREATE POLICY "Admins: Full access governance entities"
 ON governance_entities FOR ALL
 USING (auth.jwt() ->> 'role' = 'admin');

CREATE POLICY "Public: Select governance terms"
 ON governance_terms FOR SELECT
 USING (true);

CREATE POLICY "Admins: Full access governance terms"
 ON governance_terms FOR ALL
 USING (auth.jwt() ->> 'role' = 'admin');

--
================================================================================
-- SECTION 20: RLS POLICIES - ISSUE_GOVERNANCE_ENTITIES (Public)
--
================================================================================

CREATE POLICY "Public: Select issue-entity assignments"
 ON issue_governance_entities FOR SELECT
```

SHEMA SQL

```
 USING (true);


CREATE POLICY "Officials: Insert own entity assignments"
 ON issue_governance_entities FOR INSERT
 WITH CHECK (
  governance_entity_id = (auth.jwt() ->> 'governance_entity_id')::uuid
 );


CREATE POLICY "Officials: Update own entity assignments"
 ON issue_governance_entities FOR UPDATE
 USING (governance_entity_id = (auth.jwt() ->> 'governance_entity_id')::uuid);


CREATE POLICY "Admins: Full access issue_governance_entities"
 ON issue_governance_entities FOR ALL
 USING (auth.jwt() ->> 'role' = 'admin');


--
================================================================
======
-- SECTION 21: RLS POLICIES - EVIDENCE & SOURCES (Public)
--
================================================================
======


CREATE POLICY "Public: Select evidence sources"
 ON evidence_sources FOR SELECT
 USING (true);


CREATE POLICY "Admins: Full access evidence sources"
```

SHEMA SQL

```
 ON evidence_sources FOR ALL
 USING (auth.jwt() ->> 'role' = 'admin');

CREATE POLICY "Public: Select evidence"
 ON evidence FOR SELECT
 USING (true);

CREATE POLICY "Admins: Full access evidence"
 ON evidence FOR ALL
 USING (auth.jwt() ->> 'role' = 'admin');

--
================================================================================
======
-- SECTION 22: RLS POLICIES - RESOLUTION_CLAIMS (Public)
--
================================================================================
======

CREATE POLICY "Public: Select resolution claims"
 ON resolution_claims FOR SELECT
 USING (true);

CREATE POLICY "Officials: Insert own claims"
 ON resolution_claims FOR INSERT
 WITH CHECK (
   governance_entity_id = (auth.jwt() ->> 'governance_entity_id')::uuid
 );
```

SHEMA SQL

```
CREATE POLICY "Officials: Update own claims"
 ON resolution_claims FOR UPDATE
 USING (governance_entity_id = (auth.jwt() ->> 'governance_entity_id')::uuid);

CREATE POLICY "Admins: Full access resolution_claims"
 ON resolution_claims FOR ALL
 USING (auth.jwt() ->> 'role' = 'admin');

--
================================================================
======
-- SECTION 23: RLS POLICIES - OUTCOMES (Public)
--
================================================================
======

CREATE POLICY "Public: Select outcomes"
 ON issue_outcomes FOR SELECT
 USING (true);

CREATE POLICY "Officials: Insert own outcomes"
 ON issue_outcomes FOR INSERT
 WITH CHECK (
   governance_entity_id = (auth.jwt() ->> 'governance_entity_id')::uuid
 );

CREATE POLICY "Officials: Update own outcomes"
 ON issue_outcomes FOR UPDATE
 USING (governance_entity_id = (auth.jwt() ->> 'governance_entity_id')::uuid);
```

SHEMA SQL

```
CREATE POLICY "Admins: Full access issue_outcomes"
 ON issue_outcomes FOR ALL
 USING (auth.jwt() ->> 'role' = 'admin');

--
================================================================================
-- SECTION 24: RLS POLICIES - LIFECYCLE EVENTS (Public + Append-Only)
--
================================================================================

CREATE POLICY "Public: Select lifecycle events"
 ON issue_lifecycle_events FOR SELECT
 USING (true);

-- Lifecycle events INSERT restricted to backend service_role key.
-- No client-side INSERT policy. No citizen/official INSERT allowed.

--
================================================================================
-- SECTION 25: DOCUMENTATION & CONFIGURATION
--
================================================================================

--
================================================================================
```

SHEMA SQL

```
-- CONFIDENCE SCORE DISPLAY CONVENTION
--
================================================================================
--
-- All confidence, credibility, and reliability scores in the schema are stored
-- as NUMERIC(3,2) with range 0.00 to 1.00 (normalized 0-1 scale).
--
-- APPLICATION DISPLAY CONVERSION:
--   Database: 0.94   → Application: "94%"
--   Database: 0.87   → Application: "87%"
--   Database: 0.65   → Application: "65%"
--
-- APPLIES TO COLUMNS:
--   • issues.confidence_score
--   • issues.momentum_score
--   • evidence_sources.base_credibility_score
--   • evidence.ers_score
--   • evidence.semantic_match_score
--   • issue_lifecycle_events.confidence_score_at_event
--   • issue_lifecycle_events.momentum_score_at_event
--
-- BACKEND CONVERSION EXAMPLES (TypeScript):
--
--   const displayConfidence = (dbScore: number): string => {
--     return Math.round(dbScore * 100) + '%';
--   };
--   // 0.94 → "94%"
--
```

SHEMA SQL

```
--   const parseConfidence = (userInput: string): number => {
--     const percent = parseInt(userInput.replace('%', ''), 10);
--     return percent / 100;
--   };
--   // "94%" → 0.94
--


--
================================================================================
------
-- REPORT RECLUSTERING LOGIC
--
================================================================================
------
--
-- When AI re-clusters reports (moves a report from Issue A to Issue B):
--
-- FLOW:
--   UPDATE reports SET issue_id = NEW_ISSUE_ID WHERE id = REPORT_ID
--   ↓
--   Trigger: update_issue_on_report_link() fires
--   ↓
--   IF TG_OP = 'UPDATE':
--     - Issue A: report_count decremented (safely by GREATEST(..., 0))
--     - Issue B: report_count incremented
--   ↓
--   Both issues.updated_at refreshed
--
-- SAFEGUARD:
```

SHEMA SQL

```
--   IS DISTINCT FROM ensures we don't double-count or skip moves
--   when issue_id NULL ↔ NULL (no change) or NULL → Issue (new link)
--
-- FUTURE IMPROVEMENT (Phase 2+):
--   Consider nightly reconciliation job:
--    UPDATE issues SET report_count = (
--      SELECT COUNT(*) FROM reports WHERE issue_id = issues.id AND archived = false
--    )
--   Ensures counts stay accurate if triggers ever fail or are bypassed.
--


--
================================================================================
-- TIME-WINDOWED REPORT COUNTS (MVP DESIGN)
--
================================================================================
--
-- REMOVED: issues.reports_24h, issues.reports_7d, issues.reports_30d
--
-- RATIONALE:
--   Stored counters = maintenance complexity + trigger overhead
--   Derived queries = simpler + fresh data
--
-- MVP APPROACH: Compute on-the-fly in ranking/sorting queries
--
-- EXAMPLE QUERY (compute reports in last 24h):
--
```

SHEMA SQL

```
--  SELECT
--    issues.id,
--    issues.title,
--    issues.confidence_score,
--    COUNT(DISTINCT reports.id) FILTER (
--      WHERE reports.created_at > NOW() - INTERVAL '24 hours'
--    ) AS reports_24h
--  FROM issues
--  LEFT JOIN reports ON reports.issue_id = issues.id AND reports.archived = false
--  WHERE issues.is_active = true
--  GROUP BY issues.id, issues.title, issues.confidence_score
--  ORDER BY reports_24h DESC;
--
-- INDEXING STRATEGY:
--  Existing indexes support this:
--  • idx_reports_issue: JOIN performance
--  • idx_reports_location_created: created_at filter
--  • idx_issues_active: WHERE is_active filter
--
-- PHASE 2+ UPGRADE:
--  If ranking performance becomes bottleneck, add:
--    CREATE MATERIALIZED VIEW issue_rankings_24h AS ...
--    REFRESH MATERIALIZED VIEW CONCURRENTLY issue_rankings_24h (nightly job)
--  Then query the view instead of computing live.
--


--
================================================================
======
```

SHEMA SQL

```
-- GOVERNANCE MEMORY: IMMUTABILITY & PERMANENCE
--
================================================================================
--
-- THREE LAYERS OF PROTECTION:
--
-- 1. Prevent UPDATE (governance_terms)
--    TRIGGER: prevent_governance_terms_update()
--    Reason: History cannot change. Election cycles are immutable facts.
--
-- 2. Prevent DELETE (governance_terms)
--    TRIGGER: prevent_governance_terms_delete()
--    Reason: History must be retained forever. Deletions destroy accountability.
--
-- 3. Append-Only (issue_lifecycle_events)
--    TRIGGERS: prevent_lifecycle_events_update(), prevent_lifecycle_events_delete()
--    Reason: Audit trail must be tamperproof. State transitions are permanent.
--
-- GOVERNANCE MEMORY PRESERVED:
--   • Who ruled when (governance_terms)
--   • What issues were recognized and claimed resolved (resolution_claims)
--   • When status changed and why (issue_lifecycle_events)
--   • What evidence supported claims (evidence, resolution_claims.evidence_id)
--
-- SURVIVES:
--   ✓ Elections (governance_terms new record per term, never deleted)
--   ✓ Government denial (resolution_claims.is_valid tracks reality vs claims)
--   ✓ Pressure to hide (immutable triggers prevent erasure)
```

SHEMA SQL

```
--  ✓ Elections transitions (new term appended, old term preserved)
--


--
================================================================================
------
-- CITIZEN REPORT TRACKING
--
================================================================================
------
--
-- citizens.report_count is now maintained via trigger.
--
-- FLOW:
--  Citizen submits report
--   ↓ INSERT reports (citizen_id, ...)
--   ↓ TRIGGER: update_citizen_activity_on_report()
--   ↓ UPDATE citizens SET last_activity_at = NOW(), report_count = report_count + 1
--
-- USE CASES:
--  • Rank active reporters by contribution count
--  • Widget: "You've submitted X reports"
--  • Dashboard: "Top 10 reporters" table
--
-- ACCURACY:
--  Trigger-maintained counter.
--  Can be verified/reconciled nightly:
--   UPDATE citizens SET report_count = (
```

SHEMA SQL

-- SELECT COUNT(*) FROM reports WHERE citizen_id = citizens.id AND archived = false

--)

--

--
==========

-- SERVICE_ROLE AUTHORIZATION & BACKEND INTEGRATION

--
==========

--

-- PATTERN: Backend AI service uses Supabase service_role key

--

-- const supabase = createClient(SUPABASE_URL, SERVICE_ROLE_KEY);

--

-- BEHAVIOR:

-- service_role key automatically BYPASSES ALL RLS POLICIES.

-- No JWT 'role' claim validation needed.

-- No policies check service_role requests.

--

-- WORKFLOW:

-- Backend (service_role key)

-- ↓

-- POST /api/cluster-reports (server-side API route)

-- ↓

-- Supabase client with service_role key

-- ↓

SHEMA SQL

```
--   Bypasses all RLS policies
--    ↓
--   INSERT issues, issue_updates, evidence, lifecycle_events
--    ↓
--   Triggers execute (counters, timestamps, activity tracking)
--    ↓
--   Public citizens see aggregated issues via SELECT policies
--
-- SECURITY:
--   • service_role key NEVER exposed to frontend
--   • API route validates backend request (session, rate limit, etc.)
--   • Citizens cannot directly access service_role functionality
--   • All AI actions logged in issue_lifecycle_events with triggered_by enum
--
-- This is the Supabase-recommended pattern for backend services.
-- No custom "system" or "ai_service" role claim needed.
--


--
================================================================
======
-- RLS POLICY VERIFICATION CHECKLIST
--
================================================================
======
--
-- ✓ Citizens:
--   - SELECT own reports, media, conversations
--   - INSERT own reports, media
```

SHEMA SQL

```
--   - Cannot INSERT/UPDATE/DELETE governance, issues, outcomes, claims
--
-- ✓ Officials (governance_entity_id in JWT):
--   - SELECT all issues, updates, claims, outcomes (public)
--   - INSERT/UPDATE own entity updates, claims, outcomes
--   - Cannot access citizen data
--
-- ✓ Admins (role='admin' in JWT):
--   - Full access (ALL) on all tables
--
-- ✓ Service Role (bypass):
--   - Bypasses RLS entirely (no policies needed)
--   - Used by Twilio webhook, AI clustering, scheduled jobs
--
-- ✓ Public (anon, no auth):
--   - SELECT issues, issue_updates, outcomes, evidence, governance (public)
--   - Cannot INSERT/UPDATE/DELETE anything
--
-- ✓ Append-Only (issue_lifecycle_events):
--   - Public SELECT
--   - INSERT via service_role only
--   - No UPDATE/DELETE allowed (triggers enforce)
--


--
================================================================================
======
-- FINAL CORRECTIONS APPLIED
```

SHEMA SQL

```
--
================================================================================
--
-- CRITICAL BUGS FIXED:
--  ✓ citizens.report_count now incremented on report insert via trigger
--  ✓ governance_terms now protected from deletion (Governance Memory)
--  ✓ reports → issues FK constraint added (data integrity)
--
-- DESIGN IMPROVEMENTS:
--  ✓ Removed reports_24h/7d/30d denormalized counters (MVP simplicity)
--  ✓ Documented on-the-fly computation via SQL window functions
--  ✓ Added path to Phase 2+ materialized views for performance
--
-- ARCHITECTURE PRESERVED:
--  ✓ Reports ≠ Issues (private → public clustering)
--  ✓ Governance Memory (immutable history, permanent records)
--  ✓ Reality Status vs Official Status (dual tracking)
--  ✓ No human gatekeepers (AI-driven + evidence-ranked)
--  ✓ INSERT RLS policies (citizens can submit)
--  ✓ Service_role bypass (backend integration clean)
--
-- SCHEMA READY FOR PRODUCTION DEPLOYMENT
--
```